GITHUB

Pull Requests

From Start to Finish

Pluralsight | GitHub Pull Requests — Complete Guide

The 6-Step PR Lifecycle

2026

What is a Pull Request?

A Pull Request (PR) is a formal proposal to merge changes from one branch into another. It is the mechanism through which you tell others about changes you have made and pushed to a branch in a repository on GitHub.

Once a PR is opened, you can discuss and review potential changes with collaborators, add follow-up commits based on feedback, and ultimately have your changes merged into the base branch — usually main.

The name comes from the idea that you are requesting the repository to pull your branch's changes into the target branch. Even when you own the repo, the PR process enforces a structured, reviewed workflow before anything reaches main.

Why Pull Requests Matter

Reason	What It Delivers
Code Quality	Every line is reviewed before it reaches production — bugs are caught early
Knowledge Sharing	The whole team sees what changed, why it changed, and how it was implemented
Audit Trail	Full permanent history of what was discussed, what changed, and who decided what
Bug Prevention	Reviewers catch issues the author missed — fresh eyes see different problems
Team Alignment	Everyone stays aware of what is being built and how the codebase is evolving
Standards Enforcement	Reviews ensure code follows team conventions, naming, and structure

The 6-Step PR Lifecycle

Every Pull Request follows a structured lifecycle:

```
CHECKOUT → COMMIT → CREATE → REVIEW → APPROVE → MERGE
```

Each step has a distinct purpose, specific actions, and best practices. Skipping or rushing any step introduces risk and reduces code quality.

Step 1 — Checkout (Branch Creation)

What It Is

Before writing a single line of code, you create a dedicated branch for your work. This completely isolates your changes from the stable main branch. Checkout refers to switching to that new branch and beginning work there.

Why This Step Exists

- main must always remain stable and deployable — in-progress work cannot live there
- Your work-in-progress must never affect other developers until it is reviewed and approved
- A branch gives the PR a clear, bounded scope — reviewers know exactly what has changed and what has not
- Multiple developers can work in parallel on separate branches without ever conflicting

How to Implement

Always Start from an Up-to-Date Main

Never branch from a stale version of main. Always pull the latest before creating your branch:

```
git switch main
git pull origin main          # get every change merged since you last
synced
```

Create and Switch to Your New Branch

```
git switch -c feature/user-login     # modern syntax — create and switch in
one command

# Equivalent older syntax (both work):
git checkout -b feature/user-login
```

Branch Naming Conventions

Good branch names tell the story at a glance. The whole team should understand the purpose of a branch without opening it.

Type	Pattern	Example
New feature	feature/short-description	feature/user-authentication
Bug fix	fix/what-is-broken	fix/login-button-null-error
Urgent hotfix	hotfix/critical-issue	hotfix/payment-gateway-crash
Documentation	docs/what-docs	docs/update-api-reference

Type	Pattern	Example
Code refactor	refactor/what	refactor/database-layer
Experiment	experiment/what	experiment/graphql-migration

Key Rules for Branch Creation

- Use lowercase with hyphens — no spaces, no capitals, no underscores
- Keep names short but descriptive — someone reading the branch name should instantly understand its purpose
- Always branch from main (or develop in GitFlow) — never from another feature branch unless explicitly required
- One branch = one feature or fix — never combine unrelated changes in one branch
- Delete the branch immediately after merging — old branches are clutter

Step 2 — Commit (Making and Saving Changes)

What It Is

This is where the actual development work happens. You make your code changes, test them locally, and save them as a series of commits on your branch. Multiple commits are normal and encouraged — each commit should represent one logical, self-contained unit of change.

Why This Step Matters

- Each commit is a permanent, recoverable checkpoint in your branch's history
- Reviewers can examine commits individually to understand the progression and reasoning
- If something breaks, you can revert to a specific commit without losing everything
- Good commit messages become part of the PR's documentation and the codebase's permanent record

The Commit Cycle — How to Implement

Check What Has Changed

```
git status                    # see modified, staged, and untracked files
git diff                      # see the actual line-by-line changes
```

Stage Your Changes

```
git add src/auth/login.js     # stage one specific file (preferred for
focused commits)
git add src/auth/logout.js    # stage another specific file
git add .                     # stage ALL changed files at once
```

Commit with a Clear Message

```
git commit -m "feat: add login form validation with error states"
git commit -m "test: add unit tests for login validation"
```

Push to GitHub Regularly

```
git push origin feature/user-login   # first push — sets the remote tracking
branch
git push                             # subsequent pushes (upstream already
set)
```

Writing Great Commit Messages

A good commit message uses the conventional commit format:

```
<type>: <short summary in imperative mood — present tense>

<optional body — explain WHAT changed and WHY, not HOW>

<optional footer — reference issues: Fixes #42>
```

Commit Types

Type	When to Use
feat	A new feature or capability is being added
fix	A bug is being corrected
docs	Only documentation changes — README, comments, wiki
test	Adding or updating tests — no production code change
refactor	Code restructured without changing external behaviour
chore	Build process, tooling, or dependency updates
style	Formatting changes — whitespace, semicolons — no logic change
perf	A change that improves performance

Good vs Bad Commit Messages

Good Message	Bad Message
feat: add email validation to login form	fix stuff
fix: prevent null pointer when user has no profile image	WIP
test: add coverage for edge cases in cart calculation	asdfgh
docs: update README with Docker setup instructions	changes
refactor: extract payment logic into separate service	final fix

Best Practices for Commits

- Commit early, commit often — small commits are far easier to review and revert
- Each commit should compile and pass tests — never commit code that breaks the build
- One logical change per commit — separating concerns helps reviewers and future developers
- Use git commit --amend to fix the last commit message before pushing

- Use git rebase -i to clean up messy commits before opening the PR

Keeping Your Branch Up to Date

While you work, others merge to main. Keep your branch current to prevent large conflicts later:

```
# Option 1 — Merge (simple, preserves history)
git switch main
git pull origin main
git switch feature/user-login
git merge main

# Option 2 — Rebase (cleaner, linear history — advanced)
git switch feature/user-login
git rebase main
```

Rebasing rewrites your branch's commits on top of the latest main, creating a clean linear history. Use merge if you are unsure — it is always safe.

Step 3 — Create (Opening the Pull Request)

What It Is

Once your branch has commits and is pushed to GitHub, you open a Pull Request. This is the formal act of proposing your changes for review and eventual merge. Opening a PR does not mean the code is perfect — it means it is ready to be discussed, reviewed, and refined.

Why This Step Matters

- Makes your work visible to the entire team immediately
- Triggers automated checks — CI tests, linting, security scanning via GitHub Actions
- Starts the formal conversation and structured review process around the change
- Links the change to the issue it resolves — creating a complete audit trail
- Assigns the right reviewers and surfaces the PR to the people who need to act on it

How to Create a PR — GitHub UI

Method 1 — The Quick Way (After a Recent Push)

After pushing your branch, GitHub shows a yellow banner at the top of the repository page:

```
"feature/user-login had recent pushes — Compare & pull request"
```

Click Compare & pull request — this is the fastest path.

Method 2 — Manual

- Go to your repository on GitHub
- Click the Pull Requests tab then New Pull Request
- Set the base branch — where changes will be merged (usually main)
- Set the compare branch — your feature branch
- Click Create Pull Request

What Goes in the PR Description

A well-written PR description is as important as the code itself. It tells reviewers what changed, why it changed, and how to verify it works. Use a PR template to ensure consistency across all PRs in the repo.

```
## What does this PR do?
Adds user login with email/password including form validation,
error messages, and remember me functionality.
```

```
## Why is this change needed?
Fixes #42 -- users currently have no way to log in.

## How to test this
1. Run npm start
2. Navigate to /login
3. Try submitting empty fields -- should see validation errors
4. Try valid credentials -- should redirect to dashboard

## Checklist
- [x] Unit tests added and passing
- [x] No console errors
- [x] Tested on Chrome and Firefox
- [x] Documentation updated
```

Setting Up the PR Panel

Setting	What to Do and Why
Reviewers	Add people who must review this. CODEOWNERS auto-assigns based on files changed
Assignees	Usually yourself — marks who is responsible for driving this PR to completion
Labels	feature, bug, needs-review, do not merge — helps filter and prioritise PRs
Milestone	Links this PR to a sprint or release version for tracking progress
Projects	Adds this PR to the project board so it shows in the team's workflow
Linked Issues	Type Closes #42 in the description — auto-closes the issue when PR merges

Draft Pull Requests

If your work is not ready to be merged but you want early feedback or want to share your approach, open a Draft PR instead of a regular PR:

```
# In GitHub UI:
Click the dropdown arrow on 'Create Pull Request'
Select 'Create Draft Pull Request'
```

Draft PR	Regular PR
Cannot be merged until marked Ready for Review	Can be merged as soon as reviews and checks pass
Signals the team this is work in progress	Signals the code is ready for proper review
Still triggers CI checks and status checks	Triggers all checks normally
Great for early feedback, design discussion	Used for code that is complete and testable

PR Templates — Standardising PR Quality

Create a template at .github/pull_request_template.md to pre-fill the PR description for every new PR:

```
## Description
What does this PR do?

## Related Issue
Fixes #(issue number)

## How to Test
1.
2.

## Checklist
- [ ] Tested locally
- [ ] Documentation updated if needed
- [ ] No new warnings or errors
```

Commit and push this file once — GitHub automatically uses it for every new PR in the repo.

From the Command Line — GitHub CLI

```
# Install: https://cli.github.com
gh pr create --title "Add user login" --body "Fixes #42"
gh pr create --draft --title "WIP: Login page"
gh pr list                  # see all open PRs
gh pr status                # see PRs relevant to you
```

Step 4 — Review (Code Review Process)

What It Is

The review step is where teammates examine every line of change, ask questions, suggest improvements, and verify the code meets quality standards before it is allowed to merge. This is the most important step in the entire PR process — it is the quality gate that protects the codebase.

Why This Step is Critical

- Quality gate — problems caught in review never reach users or production
- Knowledge transfer — reviewers understand exactly what changed in the codebase
- Standards enforcement — ensures code follows team conventions, naming, and structure
- Security — reviewers spot vulnerabilities like missing input validation or exposed credentials
- Mentoring — junior developers learn from senior reviewer feedback in real context
- Shared ownership — no one person is the single point of failure for any feature

What Reviewers Look At

Area	What to Check
Correctness	Does the code actually do what it claims? Does it handle the stated requirements?
Logic	Are edge cases handled? What if user is null, list is empty, or network fails?
Security	Is user input sanitised? SQL injection risk? Exposed secrets or credentials?
Performance	Any unnecessary loops, N+1 queries, or memory leaks introduced?
Readability	Can another developer understand this code in 6 months without the author?
Tests	Are there tests? Do they cover the important cases and edge cases?
Documentation	Are complex functions, classes, or APIs documented clearly?
Standards	Does it follow the team's agreed coding style and conventions?

How to Review — GitHub UI

The Files Changed Tab

- Shows every file that was modified, added, or deleted in the PR
- Green lines = code that was added. Red lines = code that was removed
- You can expand or collapse individual files using the toggle arrows
- Click View file to see the full file in context, not just the diff

Leaving Inline Comments

- Hover over any line number — a blue + icon appears on the left
- Click the + to open a comment box for that specific line
- Type your comment, question, or suggestion
- Click Add single comment to post immediately, or Start a review to batch all your comments together

Starting a review is strongly preferred — it batches all your comments into one submission and sends only one notification to the author, rather than spamming them with a notification per comment.

Suggesting Exact Code Changes

GitHub's suggestion feature lets you propose the exact replacement code. The author can apply it with a single click — no manual editing required:

```
# In your review comment, click the +- icon to insert a suggestion block:
```suggestion
const userEmail = email.toLowerCase().trim();
```
The author sees a diff and clicks 'Commit suggestion' to apply it directly
```

Checking Out the Branch Locally

For complex changes or UI work, download the branch and test it on your own machine:

```
git fetch origin
git switch feature/user-login
npm install && npm test        # install deps and run tests locally
npm start                      # run the application and test manually
```

Review Types — How to Submit Your Review

When you are done reviewing, click Review Changes in the top right of the Files Changed tab:

Review Type	What It Means	When to Use
Comment	General observation, no approval verdict given	You have questions or minor notes but do not want to block the PR
Approve	Code is good, ready to be merged	After you are satisfied everything looks correct and meets standards
Request Changes	Issues MUST be addressed before merging	When you find bugs, security issues, logic errors, or major concerns

Etiquette for Good Reviews

For Reviewers

- Be specific — 'This will crash if user is null on line 47' not just 'this looks wrong'
- Be constructive — suggest the fix, do not just criticise. Provide the solution alongside the problem
- Distinguish blocking from non-blocking — use 'nit:' prefix for minor style suggestions that are not blockers
- Ask questions rather than demand — 'Could we use a Map here?' rather than 'Change this to a Map'
- Acknowledge good work — reviewers who only point out problems demoralise their teammates
- Review promptly — PRs sitting unreviewed for days block the whole team's progress

For the Author — Responding to Review

- Respond to every comment — even a simple 'Done' or 'Good point, fixed in latest commit'
- Push new commits to address feedback — the PR updates automatically, reviewers see the new changes
- Do not force-push during an active review — it rewrites history and confuses reviewers who are mid-review
- If you disagree, explain your reasoning professionally and discuss — do not silently ignore or comply
- Mark comments as Resolved once you have addressed them — keeps the thread clean

```
# After addressing review feedback:
git add .
git commit -m "fix: address review feedback -- add null check for user
object"
git push origin feature/user-login
# PR automatically updates — reviewers are notified of new commits
```

Step 5 — Approve (Approval and Status Checks)

What It Is

Approval is the formal, recorded sign-off that the code is ready to merge. Before the Merge button becomes active, two independent gates must both be green: the required number of human reviewer approvals, and all automated status checks passing.

Why This Step Matters

- Creates an auditable record of who verified the code, when, and on which version
- Enforces team standards through required approvals — cannot be bypassed accidentally
- Prevents accidental merges by requiring deliberate, recorded human sign-off
- CI/CD checks verify the code actually works in an isolated environment before it merges
- Protects main from untested or unreviewed code reaching production

Required Approvals — Branch Protection

Admins configure required approvals in Settings > Branches > Branch Protection Rules. The Merge button stays greyed out until all conditions are met:

```
Settings > Branches > Add Rule > Branch name pattern: main

[✓] Require pull request reviews before merging
    Required number of approvals: 2
    [✓] Dismiss stale reviews when new commits are pushed
    [✓] Require review from Code Owners

[✓] Require status checks to pass before merging
    [✓] Require branches to be up to date before merging
    Required status checks: [ci/tests, lint, security-scan]

[✓] Require conversation resolution before merging
[✓] Include administrators (admins follow rules too)
```

Status Checks — What They Are

Status checks are automated test results that appear directly on the PR page. They run via GitHub Actions or an external CI system and produce a result:

Status	Check	Meaning
PASS	CI / build (ubuntu-latest)	All tests pass on the target platform
PASS	lint / eslint	No linting errors or style violations

Status	Check	Meaning
PASS	security / CodeQL	No security vulnerabilities detected
PASS	coverage / codecov	Test coverage has not dropped below threshold
FAIL	CI / build (windows-latest)	Tests failing on Windows — PR BLOCKED until fixed

All required checks must show green before the Merge button becomes clickable. Even one failing check blocks the merge regardless of how many approvals exist.

What Approval Looks Like

- Reviewer finishes reading the full code in Files Changed
- Clicks Review Changes in the top right of the PR page
- Selects Approve and optionally writes a summary comment
- Clicks Submit review
- A green checkmark appears: '[Reviewer Name] approved these changes'
- If 2 approvals are required, both must be present before merging is allowed

Re-review After New Commits

When the author pushes new commits to address feedback, previous approvals are automatically dismissed if the protection rule 'Dismiss stale reviews when new commits are pushed' is enabled. This forces the reviewer to re-examine the updated code and approve again — ensuring no bad code can be snuck in after an approval.

```
# Author pushes fixes after review:
git add .
git commit -m "fix: address review feedback"
git push origin feature/user-login

# On GitHub: click the refresh icon next to the reviewer's name
# This sends them a notification to re-review the updated code
```

Step 6 — Merge (Merging into Main)

What It Is

The final step — incorporating the PR's changes into the base branch, usually main. Once all approvals are in and all checks pass, the PR is merged and the feature branch is retired. Anything merged to main can be deployed to production.

Why This Step is Carefully Controlled

- Anything merged to main can be immediately deployed — broken code affects real users
- The merge strategy chosen determines the clarity and structure of the permanent git history
- Branch protection rules ensure only verified, approved code can be merged
- The merge commit message becomes part of the permanent codebase record

The 3 Merge Strategies — In Detail

Strategy 1 — Merge Commit (Create a Merge Commit)

All individual commits from the feature branch are preserved exactly as they were. A new merge commit is added that has two parents — the tip of main and the tip of the feature branch.

```
main:    A -- B ----------------------------- M  (merge commit)
                  \                           /
feature:           C -- D -- E -- F -- G --
```

Advantage	Disadvantage
Complete audit trail — every commit in full context	main history can become noisy with many merge commits
Can see exactly how the feature evolved step by step	Harder to read at a glance when many branches merge

```
# Equivalent local command:
git switch main
git merge feature/user-login
```

Strategy 2 — Squash and Merge (Recommended for Most Teams)

All commits from the branch are squashed (combined) into one single, clean commit on main. The PR title and description become the commit message. The individual branch commits disappear from main's history.

```
main:    A -- B -- S  (one squashed commit containing all changes)

feature:       C -- D -- E  (squashed into S, branch deleted)
```

Advantage	Disadvantage
Very clean main history — one meaningful commit per feature or fix	Individual commits from the branch are not in main's history
Easy to read and search — git log --oneline is clean	Cannot git bisect into individual steps if needed later
git revert is simple — one commit to revert undoes the whole feature	Loses granularity of development steps on main

```
# Equivalent local command:
git switch main
git merge --squash feature/user-login
git commit -m "feat: add user login with email validation (#42)"
```

Strategy 3 — Rebase and Merge

Each commit from the branch is replayed individually onto the tip of main. The result is a linear history — it looks as if all development happened sequentially on main with no branching at all.

```
main:    A -- B -- C' -- D' -- E'  (commits replayed linearly)

feature:       C  -- D  -- E   (replayed onto main, no merge commit)
```

Advantage	Disadvantage
Perfectly linear history — easy to follow and bisect	Rewrites commit SHAs — cannot be traced back to original branch
No merge commit clutter in the log	Can create conflicts if commits do not apply cleanly
All individual commits preserved on main with context	More complex to understand and undo if issues arise

```
# Equivalent local command:
git switch main
git rebase feature/user-login
```

Choosing the Right Strategy

	Merge Commit	Squash & Merge	Rebase & Merge
History type	Full branching history	One commit per PR	Linear, all commits
main readability	Can become noisy	Very clean and readable	Clean with all detail

	Merge Commit	Squash & Merge	Rebase & Merge
Traceability	PR + commit level	PR level only	Full commit level
Revert complexity	Revert merge commit	Revert one commit	Revert multiple commits
Best for	Large features, long branches	Most teams — daily use	Teams preferring linear logs

How to Merge — GitHub UI

- Confirm all required approvals are present and all status checks are green
- Click the green Merge pull request button at the bottom of the PR page
- Click the dropdown arrow to select your merge strategy if needed
- Edit the merge commit message if using squash — this becomes the permanent commit
- Click Confirm merge to complete the merge
- Click Delete branch immediately to keep the repository clean

Auto-Merge

You can enable auto-merge on a PR — it will automatically merge the moment all required checks pass and required reviews are approved. Useful for dependency updates or straightforward changes where you know it will pass.

```
# Enable via GitHub UI:
On the PR page > click 'Enable auto-merge' > choose merge strategy > confirm
```

After Merging — Update Your Local Repository

After a PR merges, update your local main and clean up the branch:

```
git switch main
git pull origin main           # get the newly merged commit
git branch -d feature/user-login   # delete the local branch
git branch                     # confirm the branch is gone
```

Administration and Repository Permissions

The Permission Hierarchy

GitHub repositories have a clear chain of authority. Each permission level determines what a person can do throughout the entire PR lifecycle — from creating a PR to merging it.

Role	Access Level	Impact on PR Process
Read	View and clone only	Can view PRs, read code, and leave comments. Cannot approve or merge
Triage	Manage issues and PRs, no push	Can label, assign, and close PRs. Cannot push code or merge
Write	Push to non-protected branches	Can create branches, open PRs, review, and approve. Cannot merge to protected main without passing rules
Maintain	Manage repo without sensitive settings	Can merge PRs, manage labels and milestones. Cannot change branch protection rules
Admin	Full control over everything	Can bypass branch protection, change all settings, force-merge in emergencies

Branch Protection Rules — The Core Admin Tool

Branch protection rules are the most important administrative control over the PR process. They are configured per branch in Settings > Branches > Branch protection rules. Once set, these rules apply to everyone including admins (if Include administrators is checked).

Rule	Why It Matters
Require pull request before merging	Nobody can push directly to main — all changes must go through a PR
Required number of approvals	Ensures no one merges their own unreviewed code — minimum peer review enforced
Dismiss stale reviews on new commits	New commits after approval trigger re-review — cannot sneak bad code in after approval
Require review from Code Owners	Domain experts automatically required — security team reviews auth code, etc.
Require status checks to pass	CI must pass — broken code cannot merge regardless of how many approvals exist
Require branches to be up to date	Prevents merging a branch that is behind main — integration bugs caught early
Require conversation resolution	All review comments must be marked resolved — no open questions when merging
Require signed commits	GPG-signed commits only — verifies the committer's identity

Rule	Why It Matters
	cryptographically
Include administrators	Admins also follow the rules — prevents careless bypasses by privileged users
Restrict who can push	Only specific people or teams can push to main — everyone else must use PRs
Restrict force pushes	Nobody can rewrite the history of main — permanent protection of the audit trail
Restrict deletions	The main branch cannot be accidentally or maliciously deleted

CODEOWNERS — Mandatory Domain Expert Reviews

The CODEOWNERS file at .github/CODEOWNERS automatically assigns mandatory reviewers to PRs based on which files were changed. When Require review from Code Owners is enabled in branch protection, the PR cannot merge until those owners approve — regardless of how many other reviewers approved.

```
# .github/CODEOWNERS
# Syntax:  file-pattern    @username or @org/team

/src/auth/             @org/security-team   # auth changes require security
review
/src/api/              @backend-lead        # API changes require backend
lead
/src/billing/          @cto @billing-team   # billing needs senior approval
*.js                  @org/frontend-team   # all JS reviewed by frontend
team
/docs/                 @org/docs-team       # docs team notified on doc
changes
```

When a PR changes a file in /src/auth/, GitHub automatically adds the security team as required reviewers. The PR is blocked from merging until they approve — even if five other reviewers already approved.

Organization-Level Policies

At the organization level, admins can enforce policies that apply across all repositories in the organization:

Policy	What It Enforces
Default branch protection	Apply the same protection rules to all repos automatically — consistent standards
Required 2FA	All organization members must have two-factor authentication enabled

Policy	What It Enforces
Allowed merge strategies	Enforce only squash merge org-wide — disable merge commit and rebase options
Outside collaborators	Control who can be added as external contributors to any org repo
Repository creation	Control who can create new repos — prevent unauthorized repo sprawl
Repository visibility	Prevent members from making private repos public without admin approval

The Complete PR Lifecycle — End to End Summary

Putting all 6 steps together into the full workflow:

```
STEP 1 — CHECKOUT
  git switch main && git pull origin main
  git switch -c feature/your-feature
  Use meaningful branch names: feature/, fix/, hotfix/

STEP 2 — COMMIT
  Make changes, test locally
  git add . && git commit -m "feat: meaningful message"
  git push origin feature/your-feature
  Keep branch current: git merge main

STEP 3 — CREATE
  Open PR: base=main, compare=your feature branch
  Write clear title + detailed description
  Link the issue: Closes #42
  Set reviewers, labels, milestone, project
  Open as Draft if not yet ready for merge
  CI checks start automatically

STEP 4 — REVIEW
  Reviewers read Files Changed tab line by line
  Leave inline comments and code suggestions
  Author responds to every comment, pushes fixes
  Reviewer re-reviews updated commits

STEP 5 — APPROVE
  Reviewer: Review Changes > Approve > Submit review
  Required number of approvals must be met
  All status checks (CI, lint, security) must be green
  All review conversations must be resolved
  Only then is the Merge button enabled
```

```
STEP 6 — MERGE
  Choose strategy: Squash / Merge Commit / Rebase
  Confirm merge
  Delete the feature branch immediately
  git switch main && git pull origin main
```